



Load Data into a DynamoDB Table

M Mohammed Dawoud Mota

Table: ContentCatalog - Items returned (6)  **Actions**  **Create item**

Scan started on March 30, 2026, 16:38:23

< 1 > 

☐	Id (Number) ▲	Authors ▼	ContentType ▼	Difficulty ▼	Price ▼
☐	1	[{"S": "Natasha"}]	Project	Easy peasy	0
☐	2	[{"S": "NextWork"}]	Project	Easy peasy	0
☐	3	[{"S": "NextWork"}]	Project	Easy peasy	0
☐	201		Video		0
☐	202		Video		0
☐	203		Video		0



Introducing Today's Project!

What is Amazon DynamoDB?

Amazon DynamoDB is a fully managed NoSQL database service that lets me store and retrieve data really fast without having to worry about servers or maintenance. It is useful because it can handle huge amounts of traffic, scales automatically, and keeps performance consistent. Since it is non relational, it gives me a lot of flexibility with how I store my data, and I only pay for the capacity I use. For this project, it makes it easy to build tables, load data, and run queries without dealing with any complex setup.

How I used Amazon DynamoDB in this project

In this project, I used DynamoDB to create a set of tables and load sample data into them. I opened AWS CloudShell, used the AWS CLI to run the create table commands, and set the read and write capacity units so everything stayed within the Free Tier. After the tables were created, I ran the batch write command to load the JSON data files into each table. Once everything was set up, I used DynamoDB to store and organize the project's data and to see how items and attributes work in a non relational database.

One thing I didn't expect in this project was...

One thing I did not expect was how easy it was to work with DynamoDB using CloudShell. I thought setting up tables and loading data would be more complicated, but the AWS CLI commands made the whole process pretty straightforward. I also did not expect how flexible the data structure was.



Mohammed Dawoud M...

NextWork Student

nextwork.org

Seeing that every item can have different attributes without breaking anything was a nice surprise.

This project took me...

This project took me around 45 minutes to complete.



Create a DynamoDB table

Data in a DynamoDB table is organised into items, and each item has its own set of attributes. It is not like a traditional spreadsheet where every row has the same columns. In DynamoDB, every item can look different. One item might have five attributes while another only has two. The only thing every item must have is the table's partition key, which DynamoDB uses to find and group data quickly. This flexible structure is what makes DynamoDB non-relational.

An attribute is basically a piece of information about an item in a DynamoDB table. If the item is a student named Nikko, an attribute could be something like how many projects he's completed. What's cool is that every item can have its own set of attributes, they don't all have to match like in a traditional relational database. That flexibility is one of the big things that makes DynamoDB different.



Mohammed Dawoud M...

NextWork Student

nextwork.org

Table: NextWorkStudents - Items returned (1)  [Actions](#) [Create item](#)

Scan started on March 30, 2026, 16:17:37

< 1 > 

☐	StudentName (String) ▼	ProjectsComplete ▼
☐	Nikko	4



Read and Write Capacity

Read capacity units (RCUs) and write capacity units (WCUs) are the “engines” DynamoDB uses to handle how fast it can read and write data. 1 RCU lets DynamoDB perform up to 2 reads per second. 1 WCU lets DynamoDB perform 1 write per second. If I increase the number of reads or writes my table needs, DynamoDB increases the number of RCUs or WCUs to match. These units directly affect performance and cost, which is why the project has me set both to the lowest values to stay within the AWS Free Tier.

Under DynamoDB’s Free Tier, I get 25GB of storage, plus 25 Read Capacity Units (RCUs) and 25 Write Capacity Units (WCUs) every month. According to the project, this is enough to handle up to 200 million requests per month without being charged. As long as I keep my table settings low and delete my resources when I’m done, everything I’m doing in this project stays completely free.



Read/write capacity settings [Info](#)

Capacity mode
☐ On-demand
Simplify billing by paying for the actual reads and writes your application performs.

☒ Provisioned
Manage and optimize your costs by allocating read/write capacity in advance.

Read capacity
Auto scaling [Info](#)
Dynamically adjusts provisioned throughput capacity on your behalf in response to actual traffic patterns.
☐ On
☒ Off
Provisioned capacity units

Write capacity
Auto scaling [Info](#)
Dynamically adjusts provisioned throughput capacity on your behalf in response to actual traffic patterns.
☐ On
☒ Off
Provisioned capacity units



Using CLI and CloudShell

AWS CloudShell is a built-in shell environment inside the AWS Management Console that lets me run commands without installing anything on my computer. It already has the AWS CLI set up for me, so I can create and manage AWS resources just by typing commands. It also gives me some storage, so I can download files, unzip them, and even write scripts right inside the browser. It basically makes it easy to work with AWS through the command line.

AWS CLI is a command-line tool that lets me create and manage AWS resources using commands instead of clicking around in the console. It gives me a faster and more efficient way to work with AWS, especially when I want to automate tasks or run scripts. In this project, the CLI is already installed inside CloudShell, so I can use it right away to create tables and load data into DynamoDB.

I ran the AWS CLI commands to create the DynamoDB tables. These commands used `aws dynamodb create-table` along with the attribute definitions, key schema, and provisioned throughput settings for each table. Basically, I used the CLI to create the ContentCatalog, Forum, Post, and Comment tables instead of doing it manually in the console.



```
CloudShell
us-east-1

$ aws dynamodb create-table \
  --table-name ContentCatalog \
  --attribute-definitions \
  0 --attribute-name=id,AttributeType=N \
  1 --key-schema \
  2 AttributeName=id,KeyType=HASH \
  3 --attribute-definitions \
  4 --table-name ContentCatalog \
  5 --attribute-definitions \
  6 AttributeName=id,AttributeType=N \
  7 --key-schema \
  8 AttributeName=id,KeyType=HASH \
  9 --provisioned-throughput \
  10 ReadCapacityUnits=10,WriteCapacityUnits=1 \
  11 --query "TableDescription.TableStatus" \
  12 --formatting
"CREATING"
$ aws dynamodb create-table \
  --table-name Forum \
  --attribute-definitions \
  0 --attribute-name=forumname,AttributeType=S \
  1 --key-schema \
  2 AttributeName=forumname,KeyType=HASH \
  3 --provisioned-throughput \
  4 ReadCapacityUnits=10,WriteCapacityUnits=1 \
  5 --query "TableDescription.TableStatus" \
  6 --formatting
"CREATING"
$ aws dynamodb create-table \
  --table-name Post \
  --attribute-definitions \
  0 --attribute-name=forumname,AttributeType=S \
  1 --attribute-name=subject,AttributeType=S \
  2 --key-schema \
  3 AttributeName=forumname,KeyType=HASH \
  4 AttributeName=subject,KeyType=HASH \
  5 --provisioned-throughput \
  6 ReadCapacityUnits=10,WriteCapacityUnits=1 \
  7 --query "TableDescription.TableStatus" \
  8 --formatting
"CREATING"
$ aws dynamodb create-table \
  --table-name Comment \
  --attribute-definitions \
  0 --attribute-name=forumname,AttributeType=S \
  1 --attribute-name=subject,AttributeType=S \
  2 --key-schema \
  3 AttributeName=forumname,KeyType=HASH \
  4 AttributeName=subject,KeyType=HASH \
  5 --provisioned-throughput \
  6 ReadCapacityUnits=10,WriteCapacityUnits=1 \
  7 --query "TableDescription.TableStatus" \
  8 --formatting
"CREATING"
$ aws dynamodb create-table \
  --table-name Comment \
  --attribute-definitions \
  0 --attribute-name=forumname,AttributeType=S \
  1 --attribute-name=subject,AttributeType=S \
  2 --key-schema \
  3 AttributeName=forumname,KeyType=HASH \
  4 AttributeName=subject,KeyType=HASH \
  5 --provisioned-throughput \
  6 ReadCapacityUnits=10,WriteCapacityUnits=1 \
  7 --query "TableDescription.TableStatus" \
  8 --formatting
"CREATING"
```



Loading Data with CLI

I ran a CLI command in AWS CloudShell which was the one that loads the sample data into the DynamoDB tables. I used the AWS DynamoDB batch-write-item command along with the JSON files that came with the project. This command takes the data from those files and inserts it into the tables I just created.

```
nextworksampdata $ aws dynamodb batch-write-item --request-items file:///ContentCatalog.json
{
  "UnprocessedItems": {}
}
nextworksampdata $
nextworksampdata $ aws dynamodb batch-write-item --request-items file:///Forum.json
{
  "UnprocessedItems": {}
}
nextworksampdata $
nextworksampdata $ aws dynamodb batch-write-item --request-items file:///Post.json
{
  "UnprocessedItems": {}
}
nextworksampdata $
nextworksampdata $ aws dynamodb batch-write-item --request-items file:///Comment.json
{
  "UnprocessedItems": {}
}
nextworksampdata $
```



Observing Item Attributes

The screenshot shows a web interface titled "Attributes" with a table of attributes. The table has four columns: "Attribute name", "Value", "Type", and "Remove". The attributes listed are:

Attribute name	Value	Type	Remove
Id - Partition key	1	Number	
Authors	Insert a field	List	Remove
ContentType	Project	String	Remove
Difficulty	Easy peasy	String	Remove
Price	0	Number	Remove
ProjectCategory	Storage	String	Remove
Published	☒ True ☐ False	Boolean	Remove
Title	Host a Website on Amazon S3	String	Remove
URL	aws-host-a-website-on-s3	String	Remove

I checked a ContentCatalog item, which had the following attributes: Id, Authors, ContentType, Difficulty, Price, ProjectCategory, Published, Title, and URL.

I checked another ContentCatalog item, which had a different set of attributes, which were: Id, ContentType, Price, Services, Title, URL, and VideoType.



Benefits of DynamoDB

DynamoDB is more flexible because every item in a table can have its own unique set of attributes. In a relational database, every row has to follow the same structure with the same columns. DynamoDB doesn't force that. One item can have a bunch of attributes and another can have only a few, and that's totally fine. This makes it easier to store different kinds of data without having to redesign the whole table.

DynamoDB is faster because it does not have to scan through rows and columns like a relational database. Instead, it uses the partition key to jump straight to the exact item it needs. Since every item is stored based on that key, DynamoDB can find and return data almost instantly. This design removes a lot of the overhead that slows down traditional relational databases, which is why DynamoDB can handle huge amounts of traffic with very low latency.



Table: ContentCatalog - Items returned (6)  [Actions](#) [Create item](#)

Scan started on March 30, 2026, 16:38:23

< 1 > 

☐	Id (Number) ▲	Authors ▼	ContentType ▼	Difficulty ▼	Price ▼
☐	1	[{ "S" : "Natasha" }]	Project	Easy peasy	0
☐	2	[{ "S" : "NextWork" }]	Project	Easy peasy	0
☐	3	[{ "S" : "NextWork" }]	Project	Easy peasy	0
☐	201		Video		0
☐	202		Video		0
☐	203		Video		0